CLAUDE SKILL ▪ RELEASE NOTES

pdf-to-markdown

Version 2.5 | 12 October 2026

At a glance

What it is	A converter that turns a PDF of a paper, lecture notes or textbook chapter into text-only Markdown with exact LaTeX mathematics, plus one companion PDF of its figures. Every step is machine-checked.
When it runs	When you ask Claude to convert a PDF to Markdown (`.md`).
What you get	`NAME.md` and `NAME_figures.pdf`, ready to upload to a Project.
Requires	Claude with skills and code execution enabled. PyMuPDF and KaTeX are installed on first use; pdflatex is used when present.
Install	Customize → Skills (Section 6).

1 What's new in 2.5

Version 2.5 is a maintenance release. It fixes seven defects in the helper scripts that surfaced when converting a 21-page two-column APS paper with 131 numbered equations and six figures. The conversion procedure, the output format and the writing conventions are unchanged; transcriptions made with 2.4 remain valid.

Script	Fix
`prepare.py`	**Printed page numbers decoded.** Page labels stored as UTF-16 strings (a byte-order mark followed by the text) were reported undecoded, e.g. `<FEFF0031>` for page 1. They are now decoded, and a document whose labels simply repeat the PDF page numbers is reported as such.
`prepare.py`	**Equation tags found in every column.** Tags were accepted only in the right quarter of the page, so every tag in the left column of a two-column layout was missed (63 of 131 in the test paper). Text columns are now detected per page, and a tag is accepted when it is flush with the right edge of its own column. Inline references such as "Eq. (7)" at the end of a prose line are still rejected. Latin Modern math fonts (`LMMath*`) are now recognised as math fonts.
`figures.py`	**Figure crops kept inside their column.** Figures embedded as form objects often declare a bounding box far larger than what they draw, and the crop spilled into the neighbouring column; page numbers and running heads above a figure were also included. The crop is now clipped to the caption's column, excludes page numbers and running heads, and is trimmed to the pixels actually drawn.
`figures.py`	**No captions from running text.** A caption now needs a separator after its number (*FIG. 1.*, *Figure 1:*, *Fig. 1 |*), so a sentence beginning "Figure 1(a), computed for …" is no longer taken for a second Fig. 1.
`check.py`	**No false failures from missed tags.** A `\tag` whose "(N)" is in the text layer but was not detected as a tag is listed for a check by eye instead of failing the run. With the tag fix above this list is normally empty.
`verify_eqs.py`	**Every tagged equation on the sheets.** With all tags detected, no equation is left off the proofreading sheets (68 of 131 were before).
`verify_eqs.py`	**Proofreading crops show the whole equation.** Crops spanned the full text width and were bounded by lines of the other column, so they were about one line high and misaligned. A crop now covers only the equation's own column, between the nearest prose lines of that column (an equation row that starts at the margin is not mistaken for prose), with room for multi-line displays and over-wide rows.

Effect on the test paper:

	2.4	**2.5**
Equation tags detected	68 of 131	131 of 131
`check.py`	FAIL (63 false misses)	PASS
Equations on proofreading sheets	68	131
Figure crops correct without manual rework	2 of 6	6 of 6
False captions	1	0

A single-column test document gives the same tags and figure as in 2.4, plus the corrected caption handling.

> **Upgrading.** Replace the installed 2.4 with the 2.5 ZIP (Section 6). `SKILL.md` changes only where it describes what the scripts do; there is nothing new to learn and no need to reconvert existing transcriptions.

2 Overview

Mathematics extracted from a PDF's text layer is unreliable. Calligraphic, roman and bold letters merge; accents detach; sub- and superscripts flatten; integral and sum signs turn into placeholder codes. Reading every equation from page images avoids this, but repeating that work in every conversation is slow and costs a lot of context.

pdf-to-markdown does that work *once*. Claude transcribes each page from rendered images into Markdown, writing every formula, inline or displayed, as LaTeX read off the image. The result is then checked by script and proofread against the original. After conversion, Claude can read the document as clean source in any later conversation, at a fraction of the cost of the original PDF and without the risk of a corrupted text layer.

3 Outputs

Both files are named after the original PDF exactly as uploaded, with the same characters, case, spaces and underscores.

3.1 `NAME.md`: the transcription

- **Text only.** No image data is embedded, so the file stays small and fully searchable.
- **A legend on the first line** identifies the file as a transcription and explains its page markers.
- **Page markers.** Every PDF page opens with `<!-- page N -->`, or `<!-- page N (printed M) -->` when the number printed on the page differs from its position in the file. Answers drawn from the transcription can therefore cite pages. Where a paragraph runs across a page break, it is rejoined and the marker stands inline at the break.
- **Exact mathematics.** Inline math uses `$...$`. Displays use `$$...$$` with `\tag{N}` carrying the *printed* equation number, even when numbers are out of sequence. Typeface is reproduced exactly (`\mathcal`, `\mathrm`, `\boldsymbol`, upright constants where printed). Only KaTeX- and MathJax-compatible commands are used, with no custom macros.
- **Faithful to the original.** Headings, run-in paragraph heads and the document's own cross-reference style are kept. Apparent typos are reproduced, not corrected.
- **Figure placeholders.** Each figure becomes a line naming where to find it, followed by its transcribed caption.

An illustrative excerpt:

```
<!-- Transcription of example.pdf. "page N" markers give the PDF page on which the
   following text starts; "printed M" is the page number printed on that page. -->

<!-- page 1 (printed 213) -->

## 2.1 Linear response

The retarded correlation function is

$$ C^{\mathrm{R}}(t) = -\mathrm{i}\,\theta(t)\,\langle [\hat A(t), \hat B(0)] \rangle \
   tag{2.14} $$

*[Fig. 3: example_figures.pdf, page 1]*

**Fig. 3.** Real and imaginary parts of the response function.
```

3.2 `NAME_figures.pdf`: the figures

All figures are packed into one PDF at native size and in order of appearance. Each is labelled with its figure number and the document page it comes from. Packing is adaptive: figures are added to a page only while every figure keeps at least 150 dpi effective resolution when Claude views the page, so small labels stay legible at the lowest token cost. A document without figures gets no figures PDF. Text inside figures (axis labels, legends, diagram labels) is part of the image and is not transcribed. Every caption is in the Markdown, so the figures PDF can be left out of a Project when no figure needs to be seen.

4 How it works

4.1 Ground rules

- **No mathematics from the text layer.** Every formula is transcribed from the page image.
- **The text layer is a lookup, not a source.** It is used only to place strip cuts, find tags, page numbers and captions, and resolve a single doubtful prose word.
- **Mandatory zooms** for the distinctions most often misread: bars spanning several characters, tildes and hats, primes, the extent of under- and overbraces (including a leading minus sign), bold versus regular, and calligraphic versus roman versus italic.
- **Every token is written once.** Pages go to separate files, the document is joined by script, and corrections are targeted edits. The document is never re-output in full.
- **Source first.** If the LaTeX source is available, Claude converts it with pandoc and normalises the result instead of transcribing images.

4.2 Pipeline

Step	Tool	What happens
1 Prepare	`prepare.py`	Renders each page at 170 dpi as horizontal strips, cut in the gaps between lines so no line or equation is sliced. Reports per page whether it contains math, the strips' positions, the printed equation tags (found at the right edge of each text column, in single- and multi-column layouts), and the printed page number (from the PDF's page labels or a consistent header/footer offset).
2 Figures	`figures.py`	Locates each figure from its caption (*Fig. N*, *Figure N*, *FIG. N*, followed by a separator) and renders the whole region, vector art and embedded images together, at 200 dpi, kept inside the caption's column and trimmed to what is drawn. Writes a check image with the crop outlined, which Claude inspects before going on.
3 Transcribe	Claude	Views each page's strips in order and writes that page to its own Markdown file. Small or ambiguous symbols are re-rendered at 500–600 dpi.
4 Assemble	`assemble.py`	Joins the pages, rejoins paragraphs split by page breaks, adds the legend and page markers, packs the figures PDF and resolves figure placeholders.
5 Check	`check.py`	Runs the mechanical checks (Section 4.3). Claude fixes each finding with a targeted edit.
6 Proofread	`verify_eqs.py`	Builds proofreading sheets for the displayed equations; Claude compares them symbol by symbol.
7 Deliver	Claude	Presents `NAME.md` and `NAME_figures.pdf`, with a short report.

4.3 Quality checks

- `check.py` verifies that every formula parses in KaTeX, that every printed equation tag appears exactly once, that no text-layer debris (such as `(cid:NN)` codes) remains, and that every PDF page has exactly one marker, in order. A tag that is printed but was not detected as a tag is listed for a check by eye rather than failed.
- `verify_eqs.py` typesets each displayed equation from the transcription with pdflatex and places it under the crop of the same equation from the PDF, four per sheet. The crop covers the equation's own column, between the nearest prose lines. Equations that cannot be placed on a sheet are reported as problems to fix.
- Inline mathematics is not covered by the sheets. Claude re-reads it on the strips for any line with indices or accents.
- The final report states pages converted (with the printed range), figures and figures-PDF pages, check results, and any symbol left ambiguous, with its location.

5 Requirements

- A Claude plan with skills available and **code execution** enabled in Claude's settings.
- Python 3 with **PyMuPDF** and Pillow. PyMuPDF is installed with pip on first use if missing.
- **Node.js** with **KaTeX**, for the syntax check. KaTeX is installed with npm on first use if missing. These installs need the code-execution environment to reach the Python and npm package registries.
- **pdflatex** for the proofreading sheets. If it is absent, that step is skipped and Claude proofreads from the strips instead.

- **pandoc**, only when converting from LaTeX source.

6 Installation

1. **Package the skill.** The skill is a folder named `pdf-to-markdown` containing `SKILL.md` and `scripts/`. If you received it as a ZIP, use that file as it is. Otherwise, zip the folder so that the archive contains the folder itself, with `SKILL.md` directly inside it:

   ```
   zip -r pdf-to-markdown.zip pdf-to-markdown
   ```

2. **Enable code execution** in Claude's settings if it is not already on. If your organisation restricts network access from code execution, make sure the Python and npm package registries are allowed.
3. **Upload.** Open Customize → Skills, choose to upload a skill, and select the ZIP.
4. **Turn it on.** Check that *pdf-to-markdown* appears in the list and is enabled.
5. **Verify.** In a new chat, attach a short PDF with a few equations and ask *"Convert this PDF to Markdown."* Claude should read the skill, run `prepare.py`, and finish by presenting `NAME.md` (and `NAME_figures.pdf` if the document has figures).

> **Updating.** Keep only one copy installed. Remove the previous version and upload the new ZIP in its place. The installed version is recorded in the `metadata.version` field of `SKILL.md`. Transcriptions made with earlier versions remain valid files; reconvert a document only if you want it in the current output format.

7 Usage

7.1 Converting

Attach the PDF and ask for a conversion, for example:

- *"Convert this paper to Markdown."*
- *"Convert Chapter 3 (pages 41–68) of this book to Markdown."*
- *"Here is the PDF and its LaTeX source; convert it to Markdown."*

Download both files when the conversion finishes.

7.2 Using the result in a Project

Upload `NAME.md`, and `NAME_figures.pdf` if you need the figures, to the Project's knowledge. You can remove the original PDF from the Project or keep it alongside; if both are present, the transcription is used for content and the PDF only for questions about the page itself. With the companion skill **academic-pdf-reading** installed, Claude recognises the transcription automatically, quotes it as source and cites its page markers.

7.3 Budget

Conversion is the expensive step, and it happens once.

Item	Approximate cost
One 170 dpi strip	600 tokens
One page (4–5 strips)	2,500–3,000 tokens, plus zooms
One proofreading sheet (four equations)	1,500 tokens
Figure crop checks	a few hundred tokens each

A ten-page note fits comfortably in one conversation. For a long paper or a book, convert one chapter or section per conversation; Claude says so at the start.

8 Limitations

- **Captions drive figure detection.** Figures without a *Fig./Figure* caption, or whose automatic crop is wrong, are rendered by hand, which Claude does when the crop check shows a problem.
- **Figure text is not transcribed.** Questions about what a figure shows need the figures PDF.
- **Inline math is proofread on the strips**, not on the sheets, so it relies on the mandatory zooms and a re-read rather than a side-by-side comparison.
- **Oversized figures** that fall below 150 dpi even alone on a page are reported with a warning, not shrunk.
- **Born-digital PDFs are the intended input.** The scripts use the text layer for geometry (strip cuts, tags, page numbers, captions). Scanned documents without a usable text layer get less help from them.
- **Section-numbered equation tags** such as (2.14) are not detected as tags. The transcription still carries them in `\tag{2.14}` and `check.py` lists them for a check by eye, but they do not reach the proofreading sheets; Claude proofreads them on the strips.
- **Proofreading crops may include a neighbouring line** when the line just above or below an equation is very short (for example "with $z \in \mathbb{C}$."). This is harmless.
- **Long documents must be split** across conversations because of the context cost.
- **Faithful, not corrected.** Typos and inconsistencies in the original are reproduced as printed.

9 For maintainers: regression tests

When a LaTeX source exists for a test document, two scripts measure conversion quality. They work on any tool's Markdown output, not only this skill's.

```
python3 scripts/eval/mathcmp.py SOURCE.tex NAME.md [--lenient]
python3 scripts/eval/prosecmp.py SOURCE.tex NAME.md
```

`mathcmp.py` compares every display and inline formula token by token, after expanding zero-argument `\newcommand`s. The `--lenient` option also forgives upright versus italic i, d and e. Count exact matches: mean similarity stays high even when formulas are wrong. `prosecmp.py` compares the prose word by word. Remaining differences are often spelling-only (`F_{\rm rest}` versus `F_\mathrm{rest}`) and should be judged individually.

10 Working with academic-pdf-reading

The two skills are designed as a pair. **pdf-to-markdown** spends the image-reading effort once and produces a transcription in a documented format: a legend line, page markers with printed page numbers, `\tag`ged equations and figure placeholders. **academic-pdf-reading** recognises that format, reads it as source without re-verifying it against page images, opens the figures PDF for figure questions, and escalates only when the transcription is contradicted or structurally defective.

11 Package contents

File	Purpose
`SKILL.md`	The conversion procedure: ground rules, steps, writing conventions, budget.
`scripts/prepare.py`	Page strips, math detection, equation tags, printed page numbers.
`scripts/figures.py`	Figure detection, rendering and crop-check images.
`scripts/assemble.py`	Joins pages, adds the legend and page markers, packs the figures PDF.
`scripts/check.py`	KaTeX syntax, equation-tag, debris and page-marker checks.
`scripts/verify_eqs.py`	Side-by-side proofreading sheets for displayed equations.
`scripts/eval/mathcmp.py`	Regression test: formulas against a LaTeX source.
`scripts/eval/prosecmp.py`	Regression test: prose against a LaTeX source.